Final report

Project Title-

Network intrusion detection for cyber security

Problem Id: 09

Mamta:2028449

Shivangi:2028495

Amrita sahoo:2028637

Section: L

Problem description

Problem description

In today's highly connected digital environment, network systems are continuously exposed to a wide range of cyber threats, including unauthorized access, data breaches, denial-ofservice attacks, and other malicious activities. As the volume and complexity of network traffic increase, traditional rule-based security systems struggle to effectively identify and respond to new or evolving attack patterns. This creates a critical need for intelligent and automated solutions that can analyze large-scale network data and detect intrusions accurately and efficiently. The problem addressed in this project is the development of a Network Intrusion Detection System (NIDS) capable of distinguishing between normal and malicious network behavior using a data-driven approach.

To solve this problem, the project utilizes raw and unprocessed network traffic data obtained from publicly available sources such as Kaggle. This data is often noisy, incomplete, and contains a mixture of numerical and categorical features, making it unsuitable for direct use in machine learning models. Therefore, a crucial part of the problem involves performing comprehensive data preprocessing, including handling missing values, removing duplicates, encoding categorical variables, and scaling numerical features. These steps ensure that the dataset becomes structured, consistent, and suitable for analysis. Once the data is properly preprocessed, a machine learning algorithm, specifically K-Nearest Neighbors (KNN), is applied to classify network connections based on similarity measures such as distance metrics.

The core objective is to build a system that can learn from historical network data and accurately predict whether a new network instance represents normal activity or a potential intrusion. This involves addressing several challenges, such as selecting the optimal number of neighbors (K) in KNN, managing imbalanced datasets where attack instances may be less frequent than normal ones, and ensuring that the model performs efficiently on large datasets. Additionally, the system must minimize false positives (normal traffic incorrectly flagged as attacks) and false negatives (attacks going undetected), as both can have serious consequences in real-world applications.

Ultimately, the problem focuses on designing a scalable and reliable intrusion detection mechanism that leverages machine learning techniques to enhance network security. The expected outcome is a robust model capable of analyzing network traffic with high accuracy, supported by a well-preprocessed dataset, and adaptable to real-world scenarios where timely and precise detection of cyber threats is essential.

Steps of implementation

The project follows a structured machine learning pipeline consisting of the following phases:

1. Data Collection / Generation-

The dataset used in this project is the NSL-KDD dataset collected from the Kaggle platform for Network Intrusion Detection in Cyber Security.

The dataset is taken from the Kaggle platform, which provides real-world datasets related to cyber attacks and normal network traffic.

The dataset contains important features such as protocol type, service, flag, source bytes, destination bytes, error rates, and label (target variable).

The dataset is downloaded in CSV format and loaded into Python for further processing and analysis.

This provides real-world network traffic records used for intrusion detection.

2. Exploratory Data Analysis (EDA)-

After loading the dataset, understanding, cleaning, and analysis of the dataset is performed to prepare it for building the Network Intrusion Detection model.

A) Data Understanding-

The dataset was explored using methods like head(), info(), and isnull().sum() to understand the structure of the dataset.

This helped in checking data types, missing values, and understanding how normal and attack records are distributed.

It also helped in identifying important features affecting intrusion detection.

B)Data Cleaning-

The dataset was cleaned to ensure accuracy and improve model performance.

New useful features such as packet size and error rate were created by combining related columns.

Unnecessary columns like source bytes, destination bytes, and error rate-related duplicate columns were removed to reduce redundancy.

Missing values were identified and rows with null values were removed.

This step ensured that incomplete and irrelevant data would not affect model training.

C) Handling Categorical Data-

Machine learning models require numerical input, so categorical features were converted into numerical values.

Features such as protocol type, service, and flag were encoded into numeric form using Label Encoding.

This made the dataset suitable for machine learning algorithms.

D) Target Variable Conversion-

The target column label was converted into binary classification format.

Normal traffic was assigned value 0 and attack traffic was assigned value 1.

This helps the model perform binary classification for intrusion detection.

E) Feature Scaling-

Feature scaling was applied using StandardScaler.

Standardization was used to bring all numerical features to the same scale.

This improves model stability, accuracy, and reduces bias caused by large feature values.

F) Saving Cleaned Dataset-

After preprocessing, the cleaned dataset was stored in a new CSV file.

This cleaned dataset is ready for training machine learning models such as KNN, Decision Tree, Random Forest, or SVM for intrusion detection.

Model Development-

In this step, the K-Nearest Neighbors (KNN) classification model was implemented to learn the relationship between selected network features and the target variable (normal traffic or attack).

A) Applying K-Nearest Neighbors (KNN)-

KNN was used as the main classification model to detect whether network traffic is normal or an intrusion.

It works by finding the nearest neighboring data points and assigning the majority class among them to the new data point.

It was implemented using KNeighborsClassifier() from sklearn.

The model was trained using the selected best K value obtained from the Elbow Method.

This helps in accurate intrusion detection and classification.

B) Final Model Training-

After selecting the best K value (K = 5), the final KNN model was trained using the training dataset.

The model learned the patterns of normal and attack traffic using the training data.

This improves prediction performance on unseen testing data.

C) Prediction on Test Data-

After training the model, predictions were made on the testing dataset.

The model classified each test record as:

Normal Traffic (0)

Attack Traffic (1)

This helps in checking how accurately the model detects network intrusions.

Model Evaluation-

The KNN model was evaluated using multiple classification metrics to measure performance.

A) Accuracy Score-

Accuracy measures the overall correctness of the model.

Higher accuracy means better model performance.

It shows how many records were classified correctly.

B) Precision Score-

Precision measures how many predicted attack records were actually attacks.

Higher precision means fewer false positive predictions.

This is important for reducing false alarms in intrusion detection.

C) Recall Score-

Recall measures how many actual attack records were correctly identified by the model.

Higher recall means fewer false negatives.

This is important because missing an attack can be dangerous in cybersecurity.

D) F1 Score-

F1 Score provides the balance between Precision and Recall.

It is useful when both false positives and false negatives are important.

Higher F1 Score indicates better overall classification performance.

Confusion Matrix and Classification Metrics-

Confusion Matrix-

Confusion Matrix shows the number of:

True Positives

True Negatives

False Positives

False Negatives

It helps in understanding correct and incorrect classifications made by the KNN model.

This provides a detailed evaluation of intrusion detection performance.

Model selection - K-Nearest

Neighbors (KNN)

KNN performed best based on:

- Highest Accuracy Score
- Better Precision Score
- Higher Recall Score
- Better F1 Score
- Strong Confusion Matrix performance
- The Elbow Method was used to identify the best value of K.

After comparing accuracy for different K values from 1 to 10, K = 5 gave the best performance.

Therefore, KNN with K = 5 was selected as the final model.

The final intrusion detection prediction was performed using the KNN model because it gave the best classification performance.

The trained KNN model predicts whether new network traffic is:

Normal Traffic (0)

Attack Traffic (1)

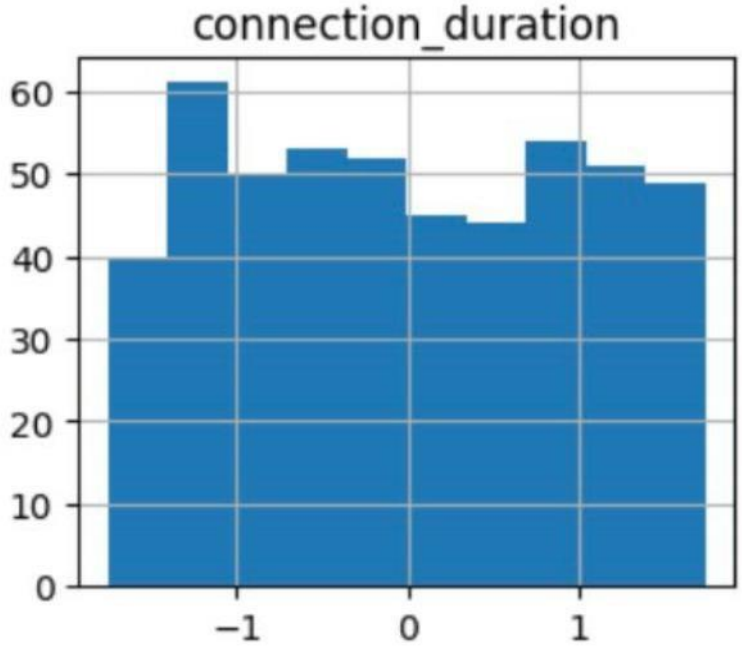
This helps in detecting possible cyber attacks in real-time network traffic.

The model uses the nearest neighboring data points to make accurate predictions for new input data.

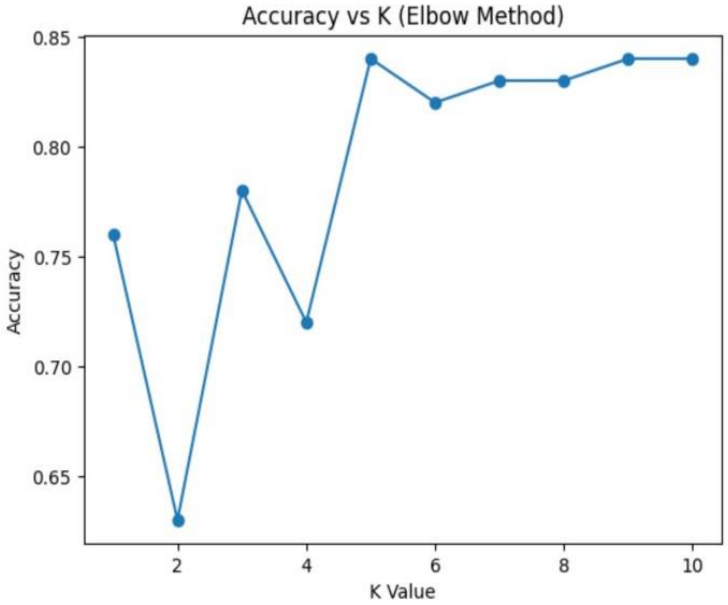
This demonstrates the practical implementation of machine learning for Network Intrusion Detection systems.

Graphs used -

Feature distribution before training



Accuracy vs k graph



Expected modules/ libraries used :

1. Pandas-
   Pandas is used for reading and handling the cleaned NSL-KDD dataset. It helps in loading the CSV file, organizing data using DataFrame, separating features and target variables, and preparing the dataset for KNN model training. It also helps in checking sample records using functions like head().

2. NumPy-
   NumPy is used for numerical computations and mathematical operations. It helps in handling numerical values, array operations, and supports calculations required during model evaluation and confusion matrix analysis. It improves the efficiency of data processing.

3. Matplotlib-
   Matplotlib is used for data visualization and graph plotting. It is used in this project for plotting: Feature Distribution Before Training Accuracy vs K Graph (Elbow Method) These graphs help in understanding data patterns and selecting the best K value for KNN.

4. Seaborn-
   Seaborn is used for advanced statistical visualization. It helps in creating better visual representations such as confusion matrix heatmaps and feature analysis plots. It improves understanding of model performance and classification results.

5. Scikit-learn-
   Scikit-learn is the main machine learning library used for implementing the KNN classification model. It provides important tools such as: train_test_split() for splitting training and testing data KNeighborsClassifier() for KNN model training accuracy_score() for model accuracy precision_score() for precision calculation recall_score() for recall calculation f1_score() for F1 Score confusion_matrix() for detailed performance evaluation It simplifies the complete machine learning pipeline for Network Intrusion Detection. • Expected Evaluation Metrics to be used: To evaluate the performance of the KNN classification model, multiple classification metrics are used to ensure accurate and reliable intrusion detection.

6. Accuracy Score-
   Accuracy measures how correctly the model predicts normal traffic and attack traffic. It shows the percentage of total correct predictions made by the model. Higher accuracy means better model performance.

7. Precision Score-
   Precision measures how many predicted attack records were actually attacks. It helps in reducing false positive predictions. Higher precision means better reliability of attack detection.

8. Recall Score-

   Recall measures how many actual attack records were correctly identified by the model. It helps in reducing false negative predictions. Higher recall is important because missing an attack can be dangerous in cybersecurity.

9. F1 Score-

   F1 Score provides the balance between Precision and Recall. It is useful when both false positives and false negatives are important. Higher F1 Score indicates better overall model performance.

10. Confusion Matrix-

    Confusion Matrix shows the number of: True Positives True Negatives False Positives False Negatives It helps in understanding correct and incorrect classifications made by the KNN model. This provides a detailed evaluation of Network Intrusion Detection performance.

Platform Required:

1. Jupyter Notebook-

Jupyter Notebook is used for writing, executing, and testing code in an interactive environment. It allows combining code, outputs, and visualizations in one place. This makes it ideal for data analysis and experimentation.

2. Python (IDLE / VS Code)-

Python is the main programming language used for implementing the project. IDEs like IDLE and VS Code provide an environment for writing, debugging, and running code efficiently. They support various libraries required for machine learning.

3. GitHub-

GitHub is used for version control and managing the project repository. It allows multiple team members to collaborate and track changes in the code. It also helps in organizing the project into phases and sharing it with others.

4. kaggle-

To download the dataset

Books and link sources

Network Intrusion Detection System Using Machine Learning.

https://indjst.org/articles/network-intrusion-detection-system-using-machine-learning?

utm

_source=chatgpt.com Kaggle (Dataset Source).

https://www.kaggle.com/datasets/programmer3/nsl-kdd-intrusion-detection-dataset

Machine learning for network intrusion detection.

https://www.geeksforgeeks.org/machine-learning/intrusion-detection-system-using-

machine-learning-algorithm